08 KUBERNETES

Module 8: Storage — Volumes, PV, PVC, StorageClass & StatefulSets

Durable data, dynamic provisioning, and stable stateful identity

VOLUMES | PV / PVC | STORAGECLASS | STATEFULSETS

From Pod-local scratch space to durable, per-replica storage.

2026 training edition

08 RECAP

From Pods to Durable Storage

01 Where we are in the course

01	02	03	04	05	06	07
Docker	**Architecture**	**Pods / Services**	**Deployments**	**Operations**	**Config**	**Scheduling**
images • runtime	control plane • kubectl	networking • lifecycle	rollouts • labels	nodes • jobs	Secrets • sidecars	resources • probes

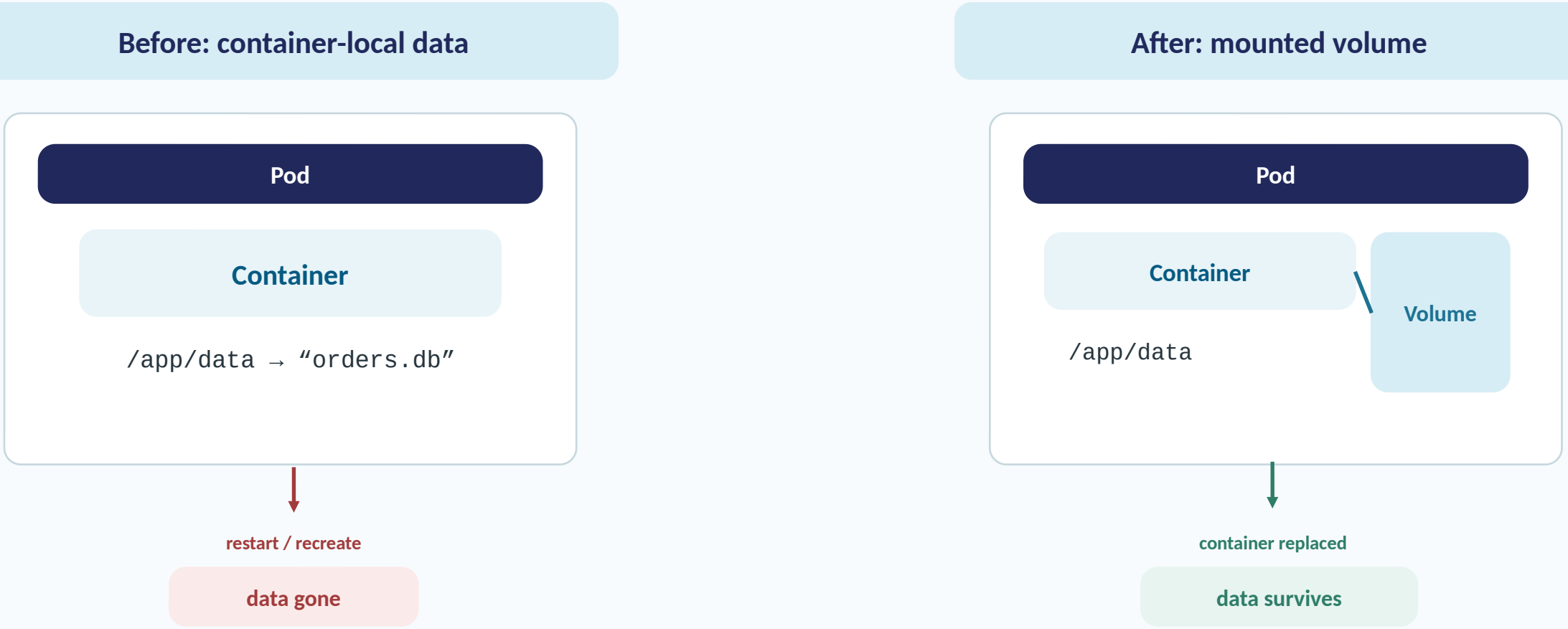
Today: storage becomes an API object, not a side effect of a container.

A	**Volume fundamentals**	emptyDir • hostPath
B	**PV / PVC / StorageClass**	static • dynamic • reclaim
C	**StatefulSets**	identity • per-replica PVCs
D	**Lab + 2026**	CSI-era storage operations

V VOLUME FUNDAMENTALS

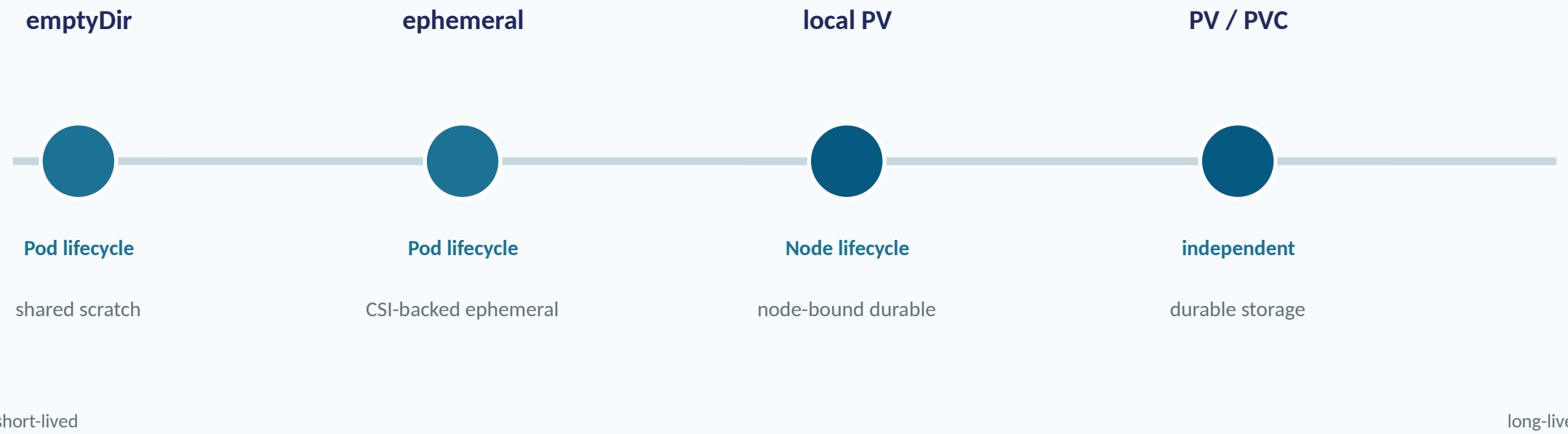
Why Do Pods Need Volumes?

A container filesystem is tied to the container instance. A volume gives the Pod an explicit place to keep or share data.



Volume Types: A Lifecycle Spectrum

Choose storage based on what must survive: a container restart, a Pod deletion, or the entire workload lifecycle.



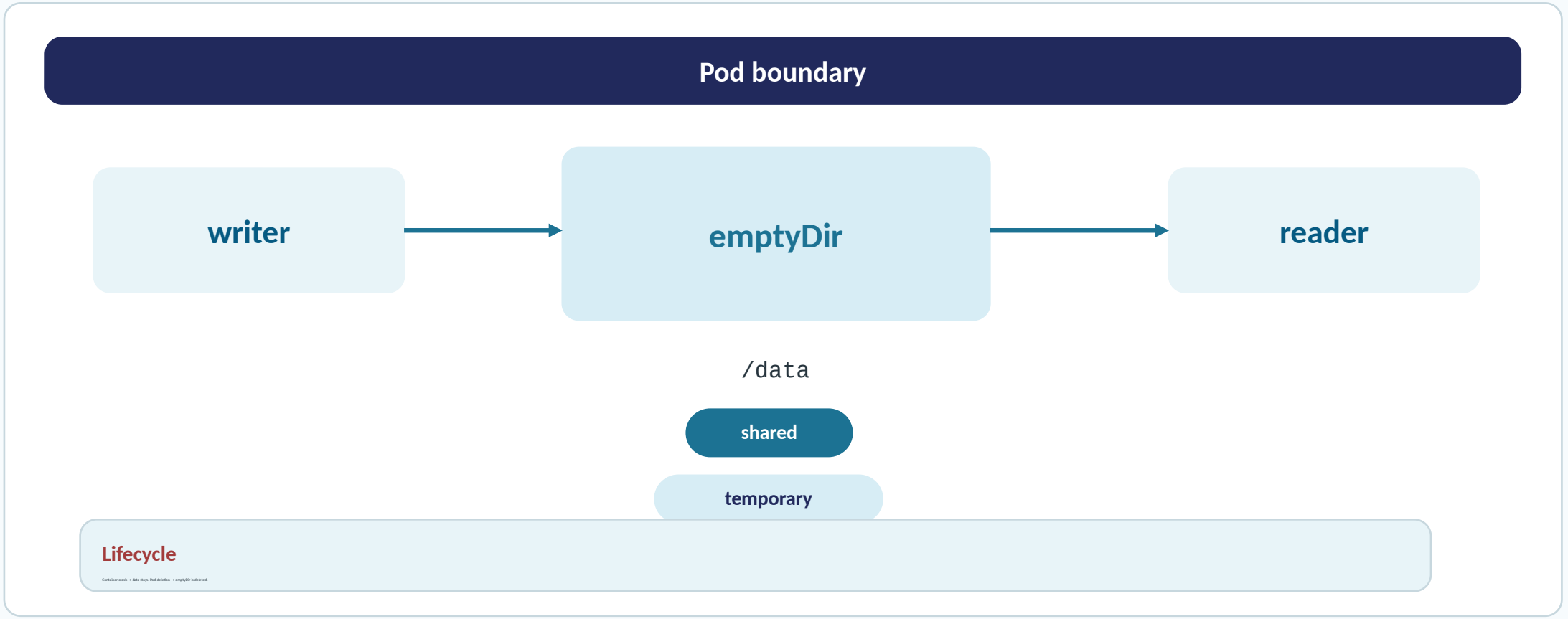
Operational rule

emptyDir is excellent for Pod-internal sharing; PV/PVC is the abstraction for durable storage. hostPath/local tie storage to nodes and reduce portability.

E VOLUME FUNDAMENTALS

emptyDir: Shared Scratch Inside One Pod

Created when the Pod is assigned to a node. Every container in that Pod can read/write it; it disappears when the Pod is removed.



E VOLUME FUNDAMENTALS

emptyDir: Full Manifest + Proof

shared-volume-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: shared-volume-pod
spec:
  volumes:
    - name: shared-data
      emptyDir:
        sizeLimit: 64Mi            # scratch space has a budget too
  containers:
    - name: writer
      image: busybox:1.37
      command: ["sh", "-c", "echo 'Hello from Writer!' > /data/message && sleep
3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
    - name: reader
      image: busybox:1.37
      # wait for the file instead of hoping the writer was first
      command: ["sh", "-c", "until [ -f /data/message ]; do sleep 1; done; cat
/data/message; sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
```

$ Terminal

```
kubectl apply -f shared-volume-
pod.yaml
kubectl logs shared-volume-pod -c
reader
```

Output

```
Hello from Writer!
```

What this proves

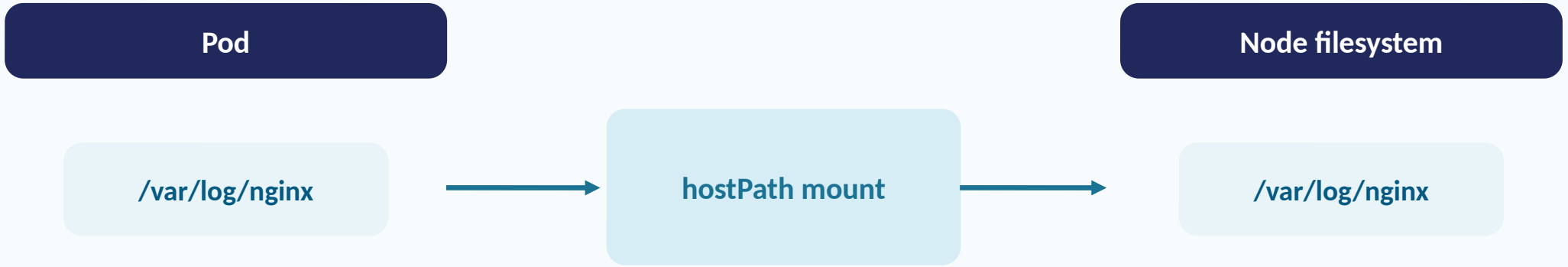
Both containers see the same files. The reader waits for the file — no race.

shared filesystem

H VOLUME FUNDAMENTALS

hostPath: Mount the Node Filesystem

hostPath exposes a directory from the underlying Node. That can be useful for node-level tooling — but it couples the workload to node-local state.



Portability warning

If the Pod lands on another node, the mounted path may be empty or different. The runbook explicitly discourages hostPath for portable production workloads.

node-local | debug / node logs | prefer PV for app data

H VOLUME FUNDAMENTALS

hostPath: Full Manifest + Mount Check

hostpath-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-volume-pod
spec:
  nodeSelector:
    kubernetes.io/hostname: lab-worker     # pinned, so we know which node
to look at
  volumes:
    - name: host-logs
      hostPath:
        path: /var/log/nginx-demo          # a folder on the NODE
        type: DirectoryOrCreate
  containers:
    - name: nginx
      image: nginx:1.28-alpine
      volumeMounts:
        - name: host-logs
          mountPath: /var/log/nginx
```

$ Terminal

```
kubectl apply -f hostpath-pod.yaml
kubectl exec hostpath-volume-pod \
  -- ls /var/log/nginx
```

Output

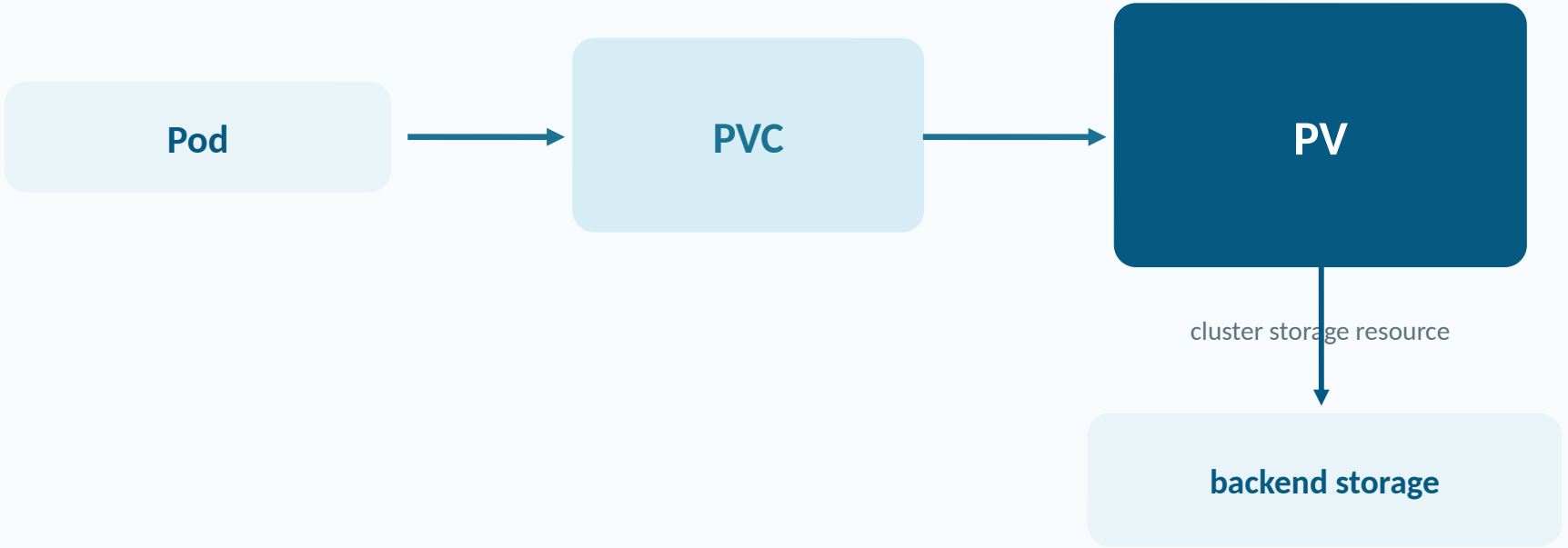
```
access.log
error.log
```

Production caution

Files stay on lab-worker only, even after the Pod is gone. Use a PVC for app data; Pod Security often blocks hostPath.

What Is a PersistentVolume (PV)?

A PV is a cluster-level storage resource with its own lifecycle. It can outlive the Pod that consumes it.



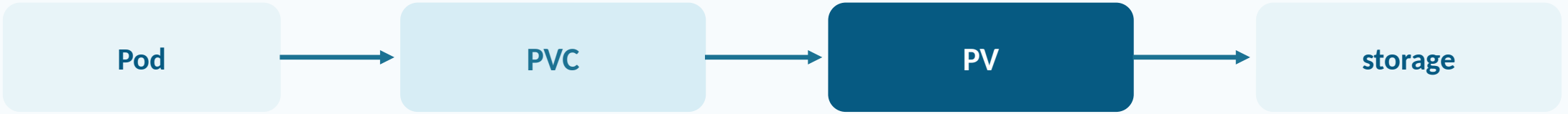
Key idea

PV is not “inside” the Pod. It is a separate API object that can remain after Pod deletion.

- capacity
- access mode
- reclaim policy

What Is a PersistentVolumeClaim (PVC)?

A PVC is the workload's storage request. Kubernetes binds it to a matching PV — or triggers dynamic provisioning when a StorageClass is used.



PVC asks for

storage size • access mode • StorageClass

PV provides

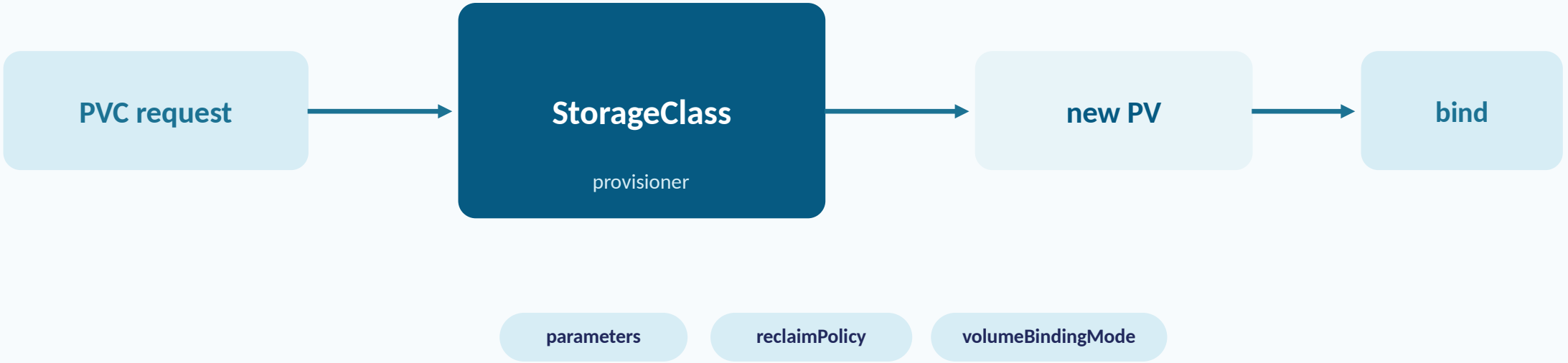
capacity • backend • lifecycle

Binding

One PVC binds to one PV. Dynamic provisioning can create the PV for the claim.

What Is a StorageClass?

A StorageClass is the recipe for dynamic provisioning: provisioner + parameters + lifecycle/binding policy.

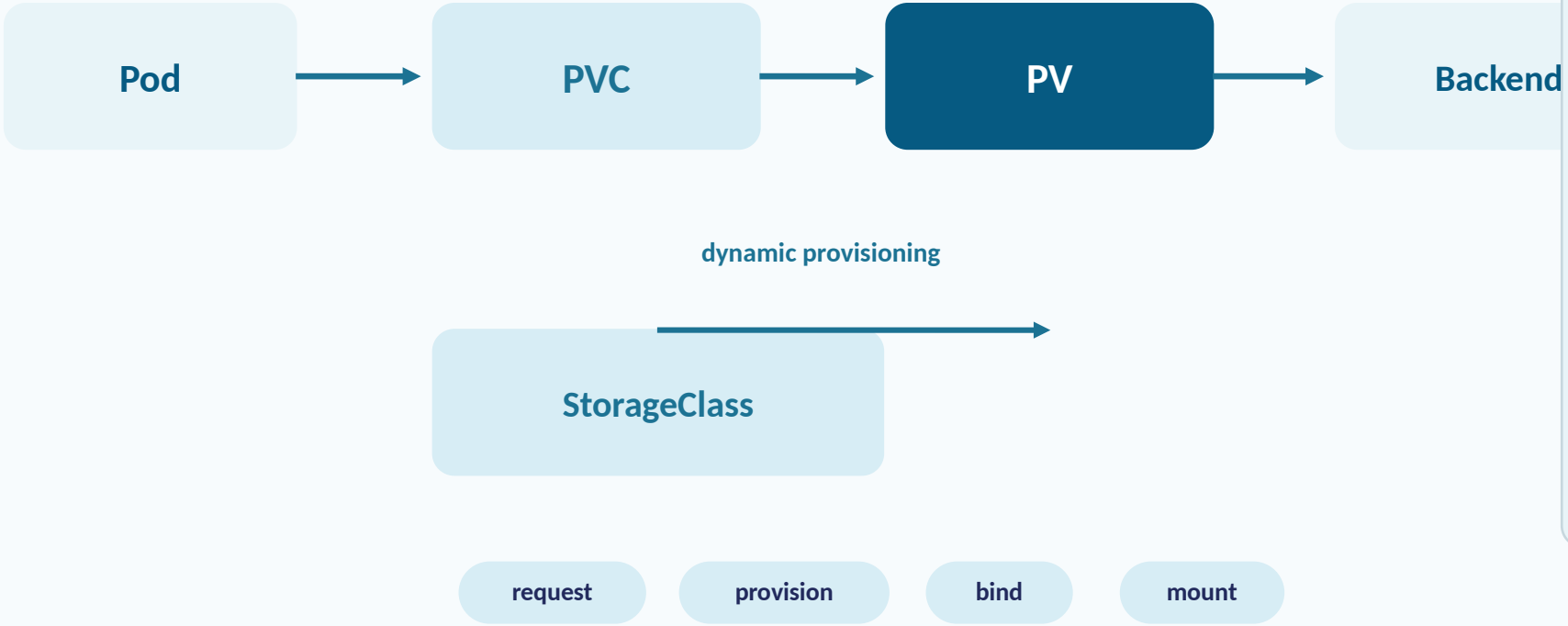


2026 pattern

Modern clusters typically use CSI provisioners directly in StorageClass objects.

PV + PVC + StorageClass: The Full Picture

Read the flow left → right: the Pod asks for storage through a PVC; the StorageClass can create the PV; the PV binds to the PVC.



Three objects

PVC = request
PV = provisioned resource
StorageClass = provisioning recipe

The Pod mounts the PVC, not the StorageClass.

SC PV / PVC / STORAGECLASS

Static vs Dynamic Provisioning

STATIC

Admin creates PVs first

PVC → matches existing PV

Useful when storage is pre-created or tightly controlled

DYNAMIC

PVC references a StorageClass

StorageClass → provisioner → PV

Modern cloud workflow; storage is created on demand

Operator lens

Dynamic provisioning reduces manual PV inventory management, but you still need to understand the StorageClass and its backend.

SC PV / PVC / STORAGECLASS

Check Available StorageClasses

$ Terminal

```
kubectl get storageclass
```

Output

```
NAME                 PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE      ALLOWVOLUMEEXPANSION
standard (default)   rancher.io/local-path   Delete          WaitForFirstConsumer   false

# a cloud example (AWS EBS CSI):   gp3   ebs.csi.aws.com   Delete   WaitForFirstConsumer   true
```

Read the columns

PROVISIONER is the driver that creates storage — kind's local-path here, a CSI driver on a cloud. RECLAIMPOLICY decides the PV after the claim is deleted; VOLUMEBINDINGMODE decides when the disk is made.

Create a PVC with Dynamic Provisioning

pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-pvc
spec:
  storageClassName: standard      # kind's default
StorageClass
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

$ Terminal

```
kubectl apply -f pvc.yaml
kubectl get pvc dynamic-pvc
```

Output

```
NAME          STATUS    STORAGECLASS
dynamic-pvc   Pending   standard

Normal  WaitForFirstConsumer
```

Why Pending?

WaitForFirstConsumer: the disk is created only when a Pod uses the claim — on that Pod's node or zone.

PVC → Pending

APP PV / PVC / STORAGECLASS

Deploy an App with the Dynamically-Provisioned PVC

deployment-with-pvc.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-storage
spec:
  replicas: 1
  strategy:
    type: Recreate                  # RWO volume: never run old and new Pod at
once
  selector:
    matchLabels:
      app: web-storage
  template:
    metadata:
      labels:
        app: web-storage
    spec:
      containers:
        - name: nginx
          image: nginx:1.28-alpine
          volumeMounts:
            - name: storage-volume
              mountPath: /usr/share/nginx/html
      volumes:
        - name: storage-volume
          persistentVolumeClaim:
            claimName: dynamic-pvc
```

$ Terminal

```
kubectl apply -f deployment-with-
pvc.yaml
kubectl get pv
```

Output

```
NAME        CAPACITY  RECLAIM  STATUS  CLAIM
pvc-7080…   1Gi       Delete   Bound
default/dynamic-pvc
```

No manual PV manifest

The first Pod triggered provisioning: the PV was created and bound automatically. Recreate keeps one Pod on the RWO volume at a time.

CHK PV / PVC / STORAGECLASS

Verify the Binding from Both Sides

$ Terminal

```
kubectl describe pvc dynamic-pvc
```

Output

```
Name:          dynamic-pvc
Status:        Bound
Volume:        pvc-7080…
StorageClass:  standard
Capacity:      1Gi
Access Modes:  RWO
```

PVC names the PV

$ Terminal

```
kubectl describe pv pvc-7080…
```

Output

```
Name:            pvc-7080…
Status:          Bound
Claim:           default/dynamic-pvc
StorageClass:    standard
Capacity:        1Gi
Reclaim Policy:  Delete
```

PV names the claim

Reclaim Policy: What Happens After PVC Deletion?

Retain

PV remains Released

Admin performs cleanup / recovery

Common choice for valuable data

Delete

PV/backend is deleted

Provisioner cleans up the volume

Common dynamic-provisioning default

Recycle

Wipe + reuse

Deprecated / no longer the modern choice

Know it historically; don’t design new workflows around it

Modern note

Current Kubernetes StorageClass reclaimPolicy supports Delete or Retain; Recycle is deprecated. The runbook's historical three-way comparison is retained here for operational context.

SS STATEFULSETS

Why StatefulSets, Not Deployments, for Stateful Apps?

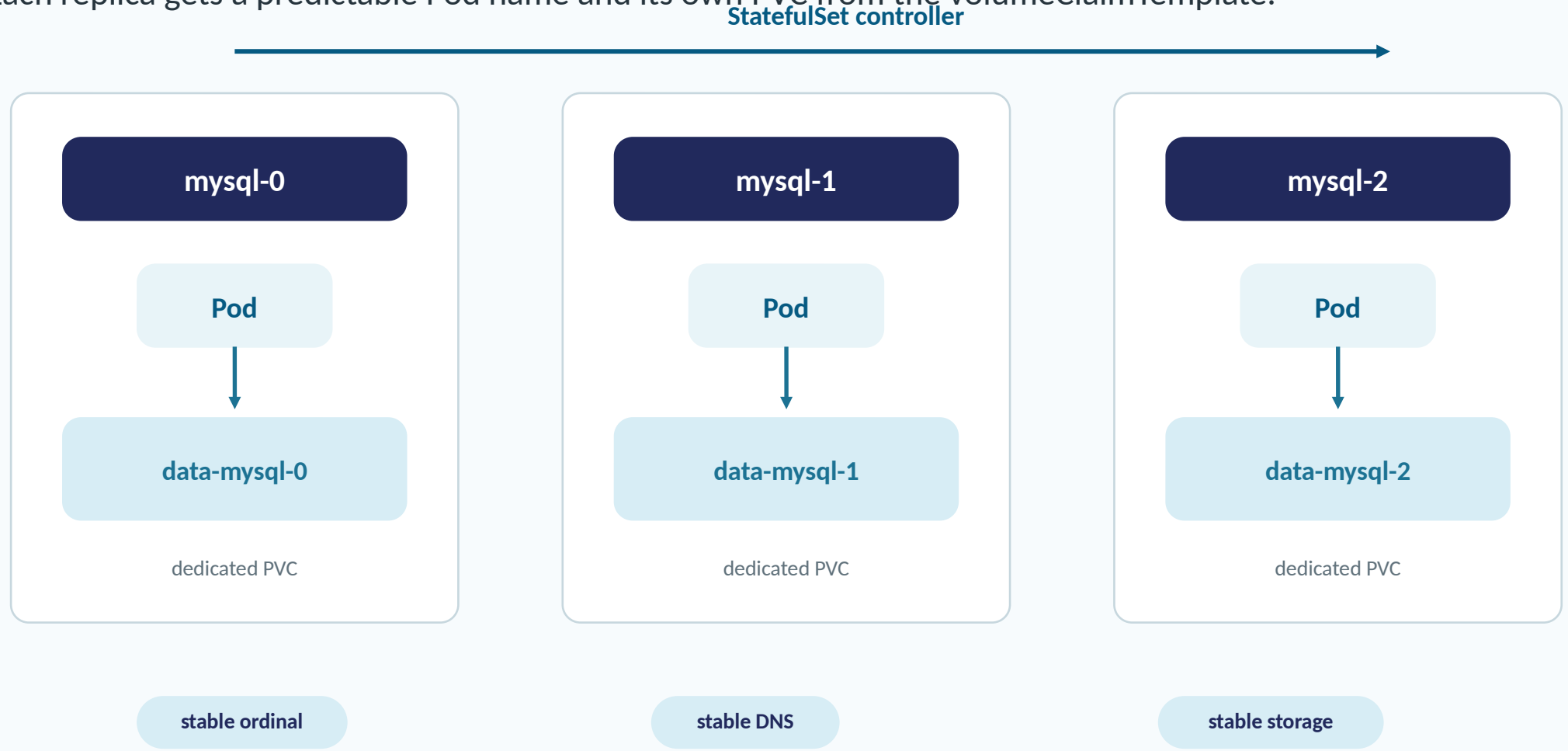
	Deployment	StatefulSet
Pod identity	interchangeable	stable: mysql-0, mysql-1, mysql-2
Network identity	not inherently stable	stable identity via headless Service
Persistent storage	often shared / external design	one PVC per replica via template
Ordering	replacement is flexible	ordered create / scale / delete

Rule of thumb

Identity + storage are part of the app contract → StatefulSet.

StatefulSet Identity = Stable Name + Dedicated PVC

Each replica gets a predictable Pod name and its own PVC from the volumeClaimTemplate.



SC STATEFULSETS

StorageClass for the StatefulSet Demo

retain-sc.yaml

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: keep-data
provisioner: rancher.io/local-path        # same driver as
"standard"
reclaimPolicy: Retain                      # keep the volume
when the claim is deleted
volumeBindingMode: WaitForFirstConsumer
```

$ Terminal

```
kubectl apply -f retain-sc.yaml
kubectl get storageclass
```

Output

```
NAME                 PROVISIONER
RECLAIMPOLICY
keep-data            rancher.io/local-path   Retain
standard (default)   rancher.io/local-path   Delete
```

Environment note

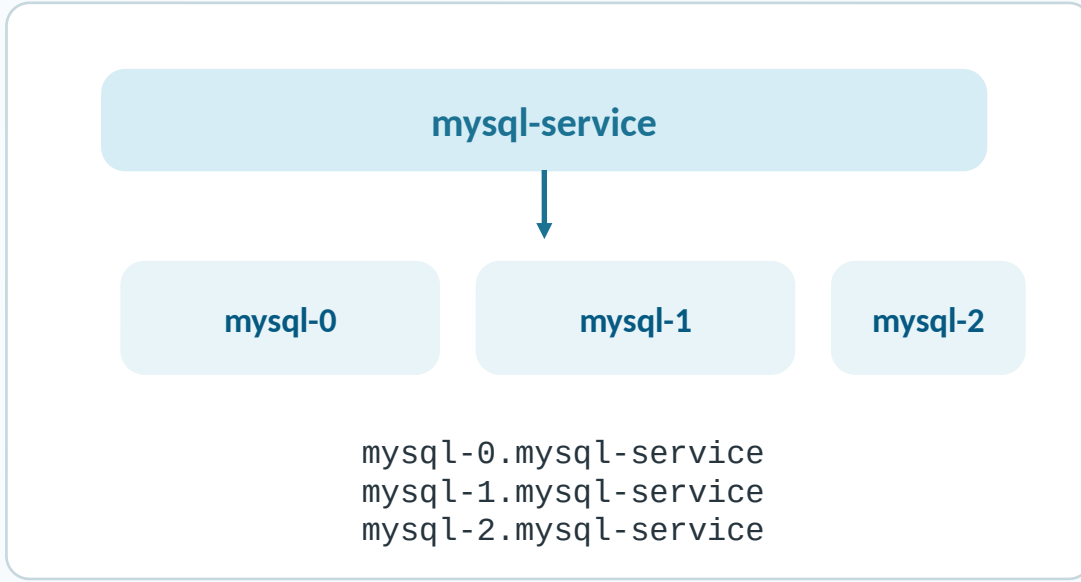
kind ships rancher.io/local-path (a folder on a node). On a cloud, use the cluster's CSI provisioner — the StorageClass API is the same.

DNS STATEFULSETS

Headless Service: Stable DNS for Stateful Pods

mysql-service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
    - name: mysql
      port: 3306
      targetPort: 3306
```



Why clusterIP: None?

A headless Service does not provide one virtual cluster IP; DNS can resolve directly to the individual StatefulSet Pod endpoints.

YAML STATEFULSETS

StatefulSet: Full MySQL Manifest

mysql.yaml (StatefulSet part — the full file adds the Secret and the headless Service)

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql-service     # headless Service -> mysql-0.mysql-service
  replicas: 3
  selector:
    matchLabels: {app: mysql}
  template:
    metadata:
      labels: {app: mysql}
    spec:
      containers:
        - name: mysql
          image: mysql:8.4.11
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef: {name: mysql-root, key: password}   # never a plain value
          readinessProbe:               # ready when the real server answers over TCP
            exec: {command: [sh, -c, 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql -h 127.0.0.1 -uroot -e "SELECT 1"']}
          resources: {requests: {cpu: 100m, memory: 384Mi}, limits: {memory: 768Mi}}
          volumeMounts: [{name: data, mountPath: /var/lib/mysql}]
  volumeClaimTemplates:
    - metadata: {name: data}
      spec:
        storageClassName: keep-data
        accessModes: [ReadWriteOnce]
        resources: {requests: {storage: 1Gi}}
```

Core idea: the template is copied per ordinal → data-mysql-0, data-mysql-1, data-mysql-2.

OBS STATEFULSETS

Deploy and Inspect: Ordered Pods + One PVC per Replica

$ Terminal

```
kubectl apply -f mysql.yaml
kubectl get pods -l app=mysql
```

Output

```
NAME      READY   STATUS    AGE
mysql-0   1/1     Running   59s
mysql-1   1/1     Running   44s
mysql-2   1/1     Running   25s
```

$ Terminal

```
kubectl get pvc -l app=mysql
```

Output

```
NAME           STATUS   CAPACITY   ACCESS MODES   STORAGECLASS
data-mysql-0   Bound    1Gi        RWO            keep-data
data-mysql-1   Bound    1Gi        RWO            keep-data
data-mysql-2   Bound    1Gi        RWO            keep-data
```

Observe

Creation is ordered and each ordinal receives its own claim. This is the storage identity that a Deployment does not provide.

EXEC STATEFULSETS

Connect to a Specific Replica

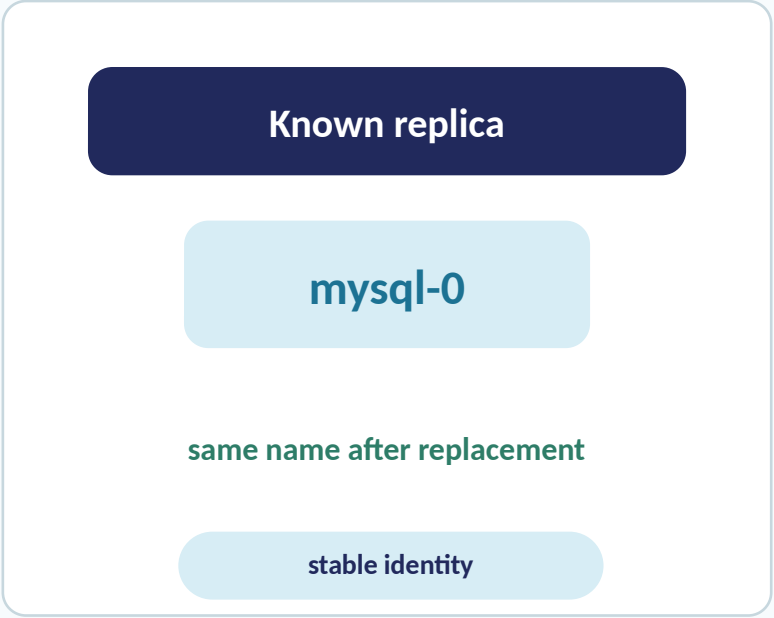
Stable names turn “which database instance?” from a discovery problem into a deterministic address.

$ Terminal

```
kubectl exec mysql-0 -c mysql -- sh -c \
  'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql -uroot \
   -e "SHOW DATABASES"'
```

Output

```
Database
information_schema
mysql
shop
…
```



Contrast

A Deployment Pod name is not an application identity. StatefulSet ordinals are intentionally stable.

SCALE STATEFULSETS

Scale a StatefulSet: Order Matters

$ Terminal

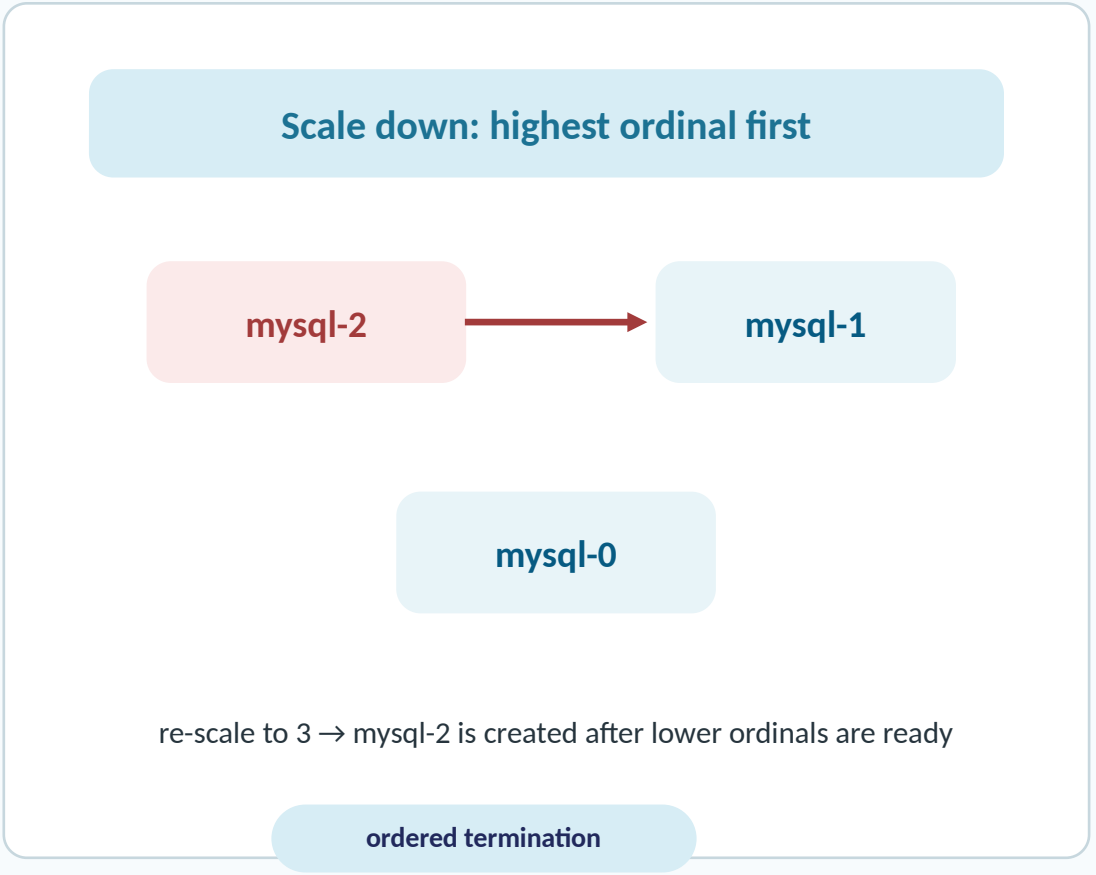
```
kubectl scale statefulset mysql --
replicas=2
```

Output

```
statefulset.apps/mysql scaled

mysql-2   Terminating
mysql-1   Running
mysql-0   Running

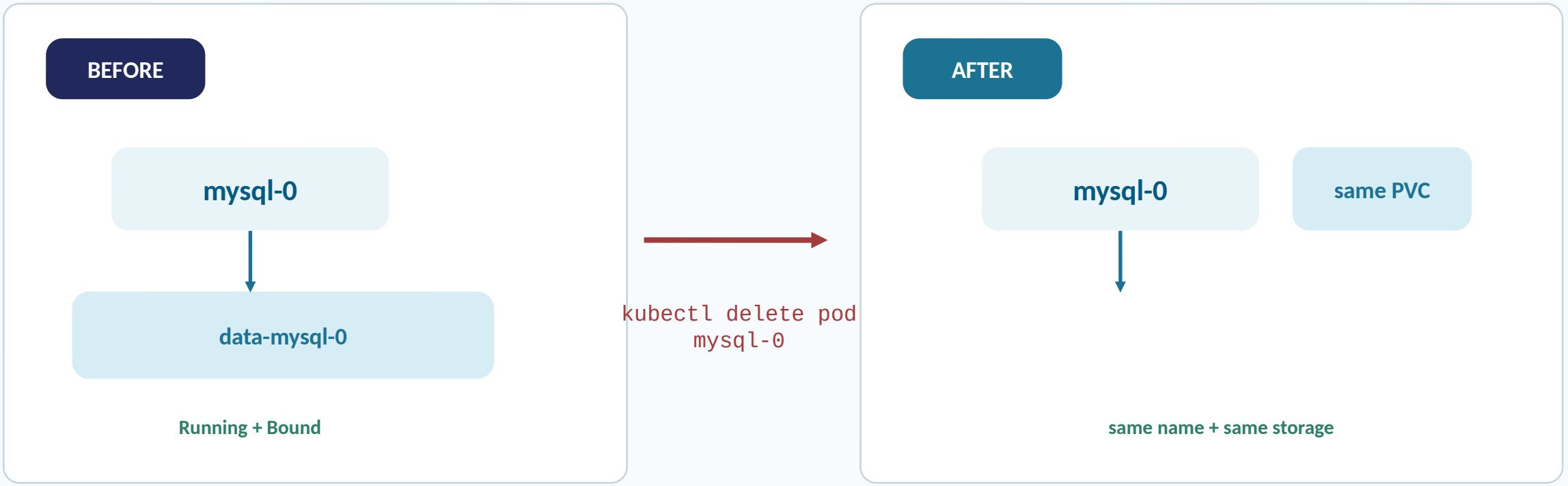
data-mysql-2   Bound   ← kept
```



AHA STATEFULSETS

Aha Moment: Delete the Pod, Keep the Identity + Data

Deleting mysql-0 deletes the Pod object — not the StatefulSet identity or its PVC. The controller recreates mysql-0 and reuses its claim.



data survives Pod deletion

PVC STATEFULSETS

Important Caveat: StatefulSet PVC Retention

The classic/default behavior is to retain PVCs when Pods disappear. Modern Kubernetes also offers an explicit retention policy for scale-down or StatefulSet deletion.

Pod deleted

PVC remains

Replacement Pod can reuse the same claim

Scale down

PVC retained by default

Optional policy can delete PVCs for scaled-down replicas

StatefulSet deleted

PVC retained by default

Optional policy can delete template-created PVCs

2026 precision

persistentVolumeClaimRetentionPolicy is stable (v1.32) and defaults to Retain. Delete removes the claim — the StorageClass reclaimPolicy still decides the volume.

LAB TRY IT YOURSELF

Hands-On Lab: Storage End to End

1	**StorageClass**	Apply retain-sc.yaml (keep-data, Retain); kind's default class is standard
2	**Dynamic PVC**	Apply pvc.yaml; it stays Pending until a Pod uses it (WaitForFirstConsumer)
3	**Deployment**	Mount dynamic-pvc in a Deployment; the PV appears and binds
4	**StatefulSet**	Apply mysql.yaml (Secret + headless Service); watch mysql-0 → mysql-1 → mysql-2
5	**Persistence**	Create shop.orders on mysql-0; delete mysql-0; the rows are still there
6	**Scale**	Scale to 2, then 3; mysql-2 gets data-mysql-2 back



LAB HANDS-ON LAB

Lab: emptyDir and hostPath — What Survives?

$ Terminal

```
kubectl apply -f shared-volume-pod.yaml
crictl stop <writer>      # from the node
kubectl delete pod shared-volume-pod
```

Output

```
Hello from Writer!
container crash → note still there
Pod deleted     → only message is left
```

$ Terminal

```
kubectl apply -f hostpath-pod.yaml
docker exec lab-worker tail -1 \
  /var/log/nginx-demo/access.log
```

Output

```
lab-worker:  "GET / HTTP/1.1" 200   ← on this node
lab-worker2: No such file or directory
Pod deleted: files stay on the node
```

Takeaway

emptyDir lives and dies with the Pod; hostPath ties data to one node. demo.html Steps 2–3

LAB HANDS-ON LAB

Lab: A Claim Waits, Then Binds

$ Terminal

```
kubectl apply -f pvc.yaml
kubectl get pvc dynamic-pvc
```

Output

```
dynamic-pvc   Pending   standard
WaitForFirstConsumer: waiting for first
consumer to be created before binding
```

$ Terminal

```
kubectl apply -f deployment-with-pvc.yaml
kubectl get pvc,pv
```

Output

```
dynamic-pvc   Bound    pvc-7080…   1Gi   RWO
pvc-7080…     Delete   Bound   default/dynamic-pvc
path: /var/local-path-provisioner/pvc-7080…
node: lab-worker2
```

Takeaway

No Pod, no disk: the volume is made where the Pod runs, and outlives every Pod. Steps 4–5

LAB HANDS-ON LAB

Lab: Access Modes and Reclaim Policies

$ Terminal

```
kubectl apply -f rwop.yaml
# 2 Pods, 1 ReadWriteOncePod claim
```

Output

```
writer-1   Running   lab-worker
writer-2   Pending
…ReadWriteOncePod access mode
already in-use by another pod
```

$ Terminal

```
kubectl delete pvc invoices   # class keep-data
kubectl get pv $PV
```

Output

```
RECLAIM POLICY   STATUS     CLAIM
Retain           Released   default/invoices

remove claimRef → Available
claim by volumeName → invoice 1001: 250 EUR
```

Takeaway

RWO = one node, RWOP = one Pod. Delete loses the data, Retain keeps it. Steps 6–8

LAB HANDS-ON LAB

Lab: A MySQL StatefulSet

$ Terminal

```
kubectl apply -f mysql.yaml
kubectl get pods,pvc -l app=mysql
```

Output

```
mysql-0   Running   59s
mysql-1   Running   44s
mysql-2   Running   25s   ← in order
data-mysql-0/1/2   Bound   keep-data
```

$ Terminal

```
nslookup mysql-0.mysql-service…
mysql -h mysql-0.mysql-service …
```

Output

```
mysql-0.mysql-service → 10.244.1.2
mysql-1.mysql-service → 10.244.2.236

mysql-0: shop.orders has 2 rows
mysql-1: no shop database   ← own disk
```

Takeaway

Stable names and one disk per replica — three separate databases, not replication. Steps 9–10

LAB HANDS-ON LAB

Lab: Delete, Scale and Retention

$ Terminal

```
kubectl delete pod mysql-0
kubectl get pod mysql-0 -o jsonpath=…
```

Output

```
before: ip=10.244.1.2  claim=data-mysql-0
after:  ip=10.244.1.5  claim=data-mysql-0
orders: 1 blue mug, 2 green tea
```

$ Terminal

```
kubectl scale statefulset mysql --replicas=2
# then with whenScaled: Delete
```

Output

```
default:            data-mysql-2 stays Bound → reused
whenScaled: Delete: data-mysql-2 deleted
its PV (Retain):    Released — data kept
```

Takeaway

The retention policy decides the claim; the reclaim policy decides the volume. Steps 11–13

Storage in 2026: CSI Is the Normal Path

The storage story has moved from built-in “in-tree” plugins toward CSI drivers and explicit StorageClass provisioners.

Older examples	**kubernetes.io/aws-ebs**	**legacy in-tree naming**
Modern StorageClass	**ebs.csi.aws.com**	**CSI driver provisioner**
GCE	**gcePersistentDisk API**	**operations redirected to pd.csi.storage.gke.io**
Ceph	**cephfs / rbd in-tree**	**removed; use CSI drivers**

Operator takeaway

When you inspect StorageClasses today, expect a CSI provisioner name. CSI Migration reached GA for major in-tree providers in v1.25, while legacy CephFS/RBD in-tree plugins were subsequently removed.

Storage in 2026: New Stable Features

All four report no alpha/beta stage on the Kubernetes v1.37 kind cluster — they are GA.

Image volumes	volumes: - image:	mount an OCI image read-only: models, content, tools
ReadWriteOncePod	accessModes: [ReadWriteOncePod]	exactly one Pod may use the volume (stable since v1.29)
VolumeAttributes	volumeAttributesClassName	change IOPS / throughput of a live volume — if the CSI driver supports it
Resize recovery	lower the PVC request	undo a failed expansion without asking an admin

Operator takeaway

Features land in stable Kubernetes, but the storage driver decides what works. kind's local-path has no expansion, snapshots or attribute classes.

LAB HANDS-ON LAB

Lab: Image Volumes — Data Shipped as an Image

$ Terminal

```
docker build -t site-content:v1 site
kind load docker-image site-content:v1
```

$ Terminal

```
kubectl apply -f image-volume-pod.yaml
kubectl exec catalogue -- wget -qO- localhost
```

Output

```
# site/Dockerfile
FROM scratch
COPY index.html /
```

Output

```
<h1>Shop catalogue v1</h1>…

touch: /usr/share/nginx/html/x:
  Read-only file system
```

Takeaway

Version data like code: read-only, cached, signed. Data the app writes still needs a PVC. Step 14

OPERATIONS

Common Pitfalls & Fast Troubleshooting

PVC Pending	No Pod yet (normal with WaitForFirstConsumer), class missing, provisioner down, or access mode not supported	`kubectl get sc; kubectl describe pvc <name>`
hostPath	Pod lands on a different node → data/path mismatch	`Prefer PV/PVC for application state`
StatefulSet PVCs	Scaling down or deleting the StatefulSet can leave retained PVCs	`kubectl get pvc; clean up intentionally`
Wrong access mode	Backend cannot satisfy RWO/RWX/RWOP requirement	`Check PVC accessModes + StorageClass/backend`

WRAP-UP

Module 8 Recap

1

Volumes

emptyDir shares scratch data within a Pod; hostPath exposes node-local paths.

2

PV / PVC

PV is durable storage; PVC is the workload request and binding handle.

3

StorageClass

Defines dynamic provisioning and storage policy.

4

Reclaim Policy

Retain vs Delete matters when a claim is removed; Recycle is deprecated.

5

StatefulSets

Stable ordinal identity + headless DNS + one PVC per replica.

6

Persistence behavior

Deleting a Pod does not delete its StatefulSet PVC; retention policies can be configured explicitly.

Mental model

Pod → PVC → PV → backend; StorageClass automates the PV step; StatefulSet repeats the storage pattern per stable replica.

08 KNOWLEDGE CHECK

9 questions — before you move on

Real storage problems with the terminal output you'd see. The answer key is on the last quiz slide.

Volumes

emptyDir • hostPath

Claims

PVC • PV • access & reclaim

StatefulSets

identity • retention

Module 8 quiz

Q1 KNOWLEDGE CHECK

Question 1 of 9 — emptyDir

$ Scenario

```
kubectl drain lab-worker …
kubectl exec api-7c9-k2 -- ls /cache
```

Question

After a node drain, the Pod's cache folder is empty. It was an emptyDir. Is something broken?

Output

```
pod/api-7c9-x4 evicted

(empty)
```

Choose one

A. The kubelet corrupted the cache
B. No — an emptyDir lives and dies with its Pod; use a PVC if it must survive
C. The drain should have copied the emptyDir
D. emptyDir needs sizeLimit to persist

Think about

Whose lifetime does emptyDir follow?

Q2 KNOWLEDGE CHECK

Question 2 of 9 — hostPath

$ Scenario

```
kubectl get pod uploads-0 -o wide
kubectl exec uploads-0 -- ls /data/uploads
```

Question

Customer uploads were saved with a hostPath volume. After a reschedule they're "gone". Best fix?

Output

```
uploads-0   Running   node-7   (was node-3)

(empty)
```

Choose one

A. Pin the Pod to node-3 forever
B. Use a bigger hostPath folder
C. Store uploads on a PersistentVolumeClaim (or object storage), not hostPath
D. Add a toleration

Think about

Where did the files stay?

Q3 KNOWLEDGE CHECK

Question 3 of 9 — Pending PVC

$ Scenario

```
kubectl get pvc reports
kubectl get storageclass standard
```

Question

A new claim has been Pending for ten minutes. No Pod uses it yet. What's going on?

Output

```
reports   Pending                         standard

VOLUMEBINDINGMODE: WaitForFirstConsumer
(no Pod uses reports yet)
```

Choose one

A. Nothing is wrong: it binds when the first Pod uses it
B. The provisioner has crashed
C. The cluster is out of disk space
D. The claim needs storageClassName: ""

Think about

What does WaitForFirstConsumer wait for?

Q4 KNOWLEDGE CHECK

Question 4 of 9 — Access modes

$ Scenario

```
kubectl get pods -l app=ledger -o wide
kubectl get pvc ledger-data
```

Question

Two replicas share one RWO claim and write the same file. Only one writer must ever exist. Fix?

Output

```
ledger-5d-a   Running   lab-worker2
ledger-5d-b   Running   lab-worker2

ledger-data   Bound   RWO
```

Choose one

A. Use ReadWriteMany
B. Add a PodDisruptionBudget
C. Use ReadWriteOncePod — a second Pod stays Pending
D. Set replicas: 2 with anti-affinity

Think about

RWO counts nodes. What counts Pods?

Q5 KNOWLEDGE CHECK

Question 5 of 9 — Reclaim Delete

$ Scenario

```
kubectl delete pvc orders-db
kubectl get pv | grep orders-db
```

Question

Someone deleted a claim and the data vanished with it. What prevents this next time?

Output

```
persistentvolumeclaim "orders-db" deleted

(nothing — the PV and its data are gone)
```

Choose one

A. A class (or PV) with reclaimPolicy: Retain for valuable data
B. Always use hostPath
C. Give the claim a bigger size
D. Label the PV "keep"

Think about

Who decides what happens to the PV?

Q6 KNOWLEDGE CHECK

Question 6 of 9 — Released PV

$ Scenario

```
kubectl get pv pvc-b71…
kubectl get pvc invoices-restored
```

Question

A new claim names the Released volume with volumeName, but stays Pending. What's missing?

Output

```
pvc-b71…   Retain   Released   default/invoices

invoices-restored   Pending
```

Choose one

A. A bigger storage request
B. Recreate the StorageClass
C. Delete the PV and start again
D. Remove the PV's old claimRef so it becomes Available

Think about

Why won't a Released PV bind?

Q7 KNOWLEDGE CHECK

Question 7 of 9 — StatefulSet identity

$ Scenario

```
kubectl delete pod mysql-0
kubectl get pod mysql-0 -o jsonpath=…
```

Question

You delete mysql-0. What does the StatefulSet give you back?

Output

```
pod "mysql-0" deleted

claim=data-mysql-0   ip=10.244.1.5 (new)
```

Choose one

A. A Pod with the same name and the same claim — the data is still there
B. A Pod named mysql-3 with an empty disk
C. Nothing until you scale up
D. A new claim data-mysql-0-2

Think about

Name, network identity, storage

Q8 KNOWLEDGE CHECK

Question 8 of 9 — Retention policy

$ Scenario

```
kubectl scale statefulset cache --replicas=3
kubectl get pvc -l app=cache
```

Question

After scaling from 5 to 3, two claims keep costing money. How do you remove them automatically next time?

Output

```
data-cache-0   Bound
data-cache-1   Bound
data-cache-2   Bound
data-cache-3   Bound   ← still billed
data-cache-4   Bound   ← still billed
```

Choose one

A. Delete the StorageClass
B. Set reclaimPolicy: Delete on the Pods
C. Use a Deployment instead
D. persistentVolumeClaimRetentionPolicy: whenScaled: Delete

Think about

StatefulSet field, not StorageClass

Q9 KNOWLEDGE CHECK

Question 9 of 9 — Two policies

$ Scenario

```
kubectl get pvc data-mysql-2
kubectl get pv pvc-452…
```

Question

whenScaled: Delete removed data-mysql-2 — but its PV still exists as Released. Why?

Output

```
Error: pvc "data-mysql-2" not found

pvc-452…   Retain   Released   default/data-mysql-2
```

Choose one

A. It's a bug in the StatefulSet controller
B. The claim is gone, but the class says reclaimPolicy: Retain, so the volume stays
C. The PV is still in use by mysql-2
D. Released PVs are deleted after 24 hours

Think about

Claim vs volume

KEY KNOWLEDGE CHECK

Answer Key

Score yourself

```
8–9  Module 9 ready
6–7  re-read what you missed
≤ 5  redo Steps 2–13
```

Tip

Every answer names a slide and a demo.html step.

Answers 1–5

Q1	B	emptyDir dies with Pod	→ slide 30 • Step 2
Q2	C	PVC, not hostPath	→ slide 30 • Step 3
Q3	A	Pending until a Pod	→ slide 31 • Step 4
Q4	C	ReadWriteOncePod	→ slide 32 • Step 6
Q5	A	reclaimPolicy: Retain	→ slide 32 • Step 7

Answers 6–9

Q6	D	remove the claimRef	→ slide 32 • Step 8
Q7	A	same name, same claim	→ slide 34 • Step 11
Q8	D	whenScaled: Delete	→ slide 34 • Step 13
Q9	B	reclaimPolicy: Retain	→ slide 34 • Step 13

Volumes & claims

StatefulSets

Next: Module 9 — RBAC & Security

Storage gives workloads durable state; next we control who can access the cluster and what they are allowed to do.

RBAC

Roles • RoleBindings • least privilege

Security

Pod Security Admission • namespaces

Identity

users • service accounts • certificates

Module 8 complete